

Anteater Poker Software Architecture Specification
Version 2.0

EECS 22L: Software Engineering Project

University of California, Irvine

Team Name:

T3

Team Members:

Jonghyun Choi: 39487186

Vinh Nguyen: 76073226

Hamza Faisal: 29338975

Richard Nguyen: 30864762

William Kuang: 58552955

Kun Liang: 61607118

Date: May 12, 2026

Table of Contents

- Glossary of Implementation Terms
- 1. Poker Client Software Architecture Overview
 - 1.1 Main Data Types & Structures
 - 1.2 Major Software Components
 - 1.3 Module Interfaces
 - 1.4 Overall Program Control Flow
 - 1.5 Automated Client: Poker Bot
- 2. Poker Server Software Architecture Overview
 - 2.1 Main Data Types & Structures
 - 2.2 Major Software Components
 - 2.3 Module Interfaces
 - 2.4 Overall Program Control Flow
- 3. Installation
 - 3.1 System Requirements / Compatibility
 - 3.2 Unpacking & Configuration
 - 3.3 Building, Compilation, & Installation
- 4. Documentation of Packages, Modules, & Interfaces
 - 4.1 Data Structures
 - 4.2 Functions & Parameters
 - 4.3 Communication Protocol
- 5. Development Plan & Timeline
 - 5.1 Task Partitions
 - 5.2 Team Member Responsibilities
- Copyright / References / Index

Glossary of Implementation Terms

- **Application Program Interface (API)**: The set of major module functions, including their parameters and return types, that allow different components of the Poker system to interact.
- **Module Hierarchy**: The structural organization of the software components, typically represented by a diagram showing how high-level modules depend on low-level utility modules.
- **Program Control Flow**: The execution sequence of the software, spanning from initialization and socket setup to the main event loop and program termination.
- **TCP/IP Socket API**: The set of system calls [e.g., socket(), bind(), listen(), connect()] used to implement network inter-process communication.
- **Communication Protocol Flags**: Specific settings or headers defined in the custom poker protocol to manage game states and message types [e.g., ENTER, BET, QUIT].
- **Abstract Data Type**: Encapsulated data structures [e.g., a Deck or Player struct] where the internal representation is hidden from the rest of the program.
- **Authoritative “Source of Truth”**: A design pattern where the Poker Server maintains sole ownership and execution of the master game state. All critical logic—including deck randomization, card distribution, hand evaluations, and pot allocations—occurs exclusively on the server. Clients operate purely as read-only presentation pipelines that cannot alter system variables independently; they must submit action requests to the server and passively render the verified updates returned over the network socket layer. This guarantees data synchronization across all concurrent sessions and eliminates client-side tampering.
- **Hierarchical Widgets (GTK)**: The parent-child data structures used in the client-side GUI to organize the visual interface.
- **Client-Server Software Architecture**: A distributed application structure that partitions tasks between the providers of a resource or server (servers) and service requesters (clients).
- **Event-Driven Control Flow**: A program execution model where the flow of the program is determined by events such as user actions (GUI button clicks) or messages arriving from a socket.
- **In-Order Reliability**: A characteristic of the TCP/IP stream sockets ensuring that data is transported without loss, duplication, or incorrect sequencing.
- **Modular Software Components**: Self-contained units of code with specific functionality [e.g., Dealer Choice Module, Game State Manager] that interact through defined APIs.
- **Unit Testing**: The practice of testing individual functions and parameters with specific test data in debug mode before full system integration.

1. Poker Client Software Architecture Overview

1.1 Main Data Types & Structures

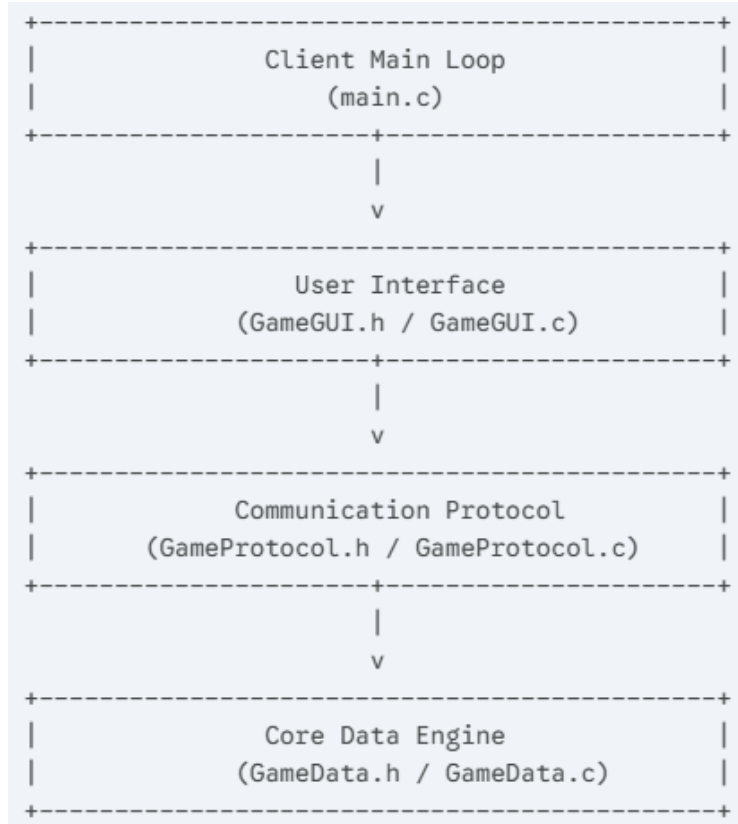
The Poker Client application serves as an event-driven user interface and network terminal that captures player inputs, displays the current visual table configuration, and dispatches validated player choices to the authoritative server process over a dedicated socket connection. To maintain strict data separation and adhere to the project's design parameters, the client does not possess standalone game-altering capabilities; it lacks hand evaluation metrics locally and cannot calculate pot distributions or alter card states independently.

Instead, the client application imports the core tracking types from `GameData.h` to allocate a local, read-only memory footprint of the Table and Player structures. This data profile is updated continuously via incoming network packets to refresh its hierarchical layout elements.

1.2 Major Software Components

The client subsystem is constructed from four core software units designed to isolate presentation layout elements from communication serialization channels:

- **Presentation Layer** (`GameGUI.c` / `GameGUI.h`): Manages the top-level window frames, 2D table drawing placeholders, and interactive button groups via the GTK 3.0 event-driven framework.
- **Network Communication Core** (`ClientNetwork`): Interfaces directly with the Linux system socket layer to instantiate TCP connections, handle stream inputs, and manage non-blocking read/write loops.
- **Protocol Translation Deck** (`GameProtocol.c` / `GameProtocol.h`): Implements safe parsing mechanics (`sscanf`, `snprintf`) to translate user actions into structured protocol strings and decode server state broadcasts.
- **Local Data Core** (`GameData.h`): Provides data type signatures (`Card`, `Player`, `Table`) to define the local, passive model engine that standardizes communication boundaries.



[Figure 1: Diagram of Client Module Hierarchy]

1.3 Module Interfaces

Communication within the client application maps to asymmetric, functional pathways that cleanly bridge the graphical layer with the background network driver:

- **Outbound Pipeline:** User-initiated mouse interactions with action buttons trigger explicit widget signals (e.g., clicking FOLD triggers an interaction callback mapped to `PLAYER_ACTION_FOLD`). The interface layer hands this context to the protocol engine, which packages the parameters into a newline-terminated ASCII string using `BuildActionMessage`. This serialized array is then forwarded to the network core to be pushed into the stream socket.
- **Inbound Pipeline:** The network processing core monitors the active socket file descriptor. Upon receiving data, it passes the raw buffer to `ParseNetworkMessage` within the protocol layer. If verified, the engine alters the local Table memory copy and invokes a graphical redraw event, triggering the GTK layout tree to update card assets, point pools, and player statuses safely.

1.4 Overall Program Control Flow

The execution life cycle of the client process steps through a synchronous bootstrapping routine before settling into an asynchronous, event-driven state:

1. **System Initialization:** The program starts inside `main.c`, directly passing runtime arguments to `InitializeGUI` where the top-level `GtkWindow` framework, drawing tables, and button sets are allocated.
2. **Network Handshake:** The network core opens a stream pipe using `connect()`. Once verified, the client outputs an explicit `ENTER` command containing the local player's alias, chosen seat index, and the session password.
3. **Event Loop Delegation:** The program triggers `ShowMainWindow()` and completely hands execution control over to the blocking, event-driven `gtk_main()` kernel thread.
4. **Asynchronous Processing:** The thread blocks continuously until a user interacts with a widget button or an inbound response packet from the server forces a visual update.
5. **Graceful Teardown:** Closing the top-level window invokes a destruction signal handler that safely alerts the protocol layer, pushes a termination notification to the server socket, releases all local layout data structures, and terminates execution safely.

1.5 Automated Client: Poker Bot

The automated agents have been refactored into embedded POSIX threads (`pthread`s) executing directly within the authoritative server process space. They establish localized TCP loopback connections (127.0.0.1) back into the parent server's listening socket.

The embedded automated agents lack any reference to the `GameGUI` subsystem or `GTK` libraries, reducing processing overhead. Each thread runs a lightweight connection loop that parses inbound server updates into a local `Table` mirror. Once the server marks that the active turn index matches the bot's seated position, the bot invokes an automated strategy matrix. This mathematical selector analyzes the hole cards alongside community cards, computes optimal wagering values, generates the corresponding `ACTION` protocol text array via the serialization engine, and transmits the move back into the socket instantly before returning to a passive listening state.

2. Poker Server Software Architecture Overview

2.1 Main Data Types & Structures

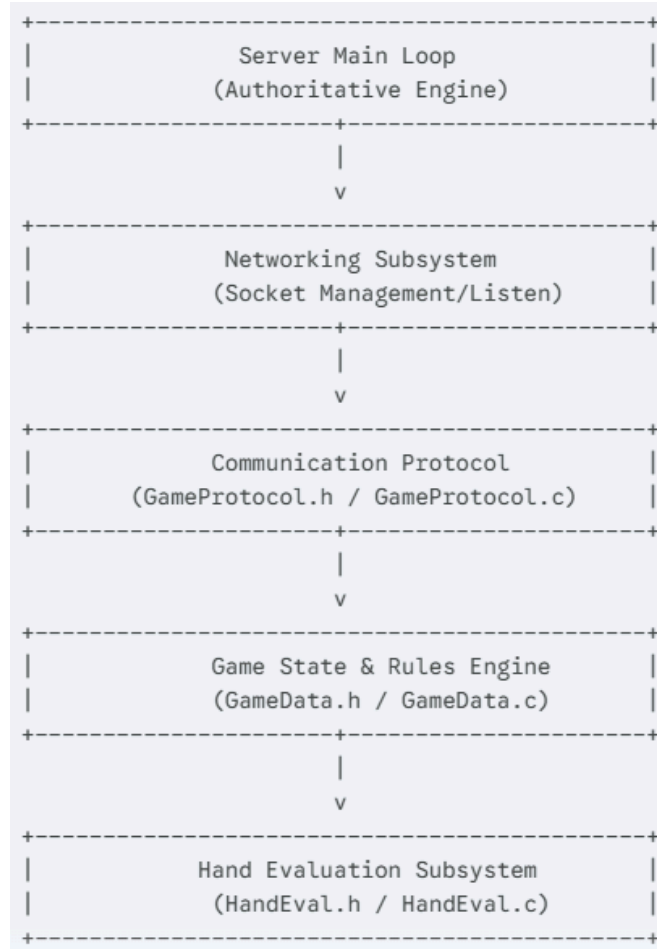
The Poker Server application acts as the definitive, host-side processing engine and authoritative "Source of Truth" for the entire system topology. To guarantee total protection against client-side memory manipulation or unauthorized game mutations, the server maintains exclusive ownership of the master game state matrix.

The structural baseline of this environment is defined in `GameData.h` by the `Table` struct, which encapsulates global parameters including the active game phase indicator (state), running point arrays (pot), and the relative tracking indices for the dealer button position and immediate turn action (dealerIdx, activeIdx). This structure contains nested tracking matrices: a static `Player` array tracking authentication keys, dynamic point balances, and active network connections ; alongside a standard `Card` layout allocated for the shared community card pool.

2.2 Major Software Components

The server application relies on an isolated, modular component stack to cleanly decouple network transport pipelines from mathematical hand evaluation mechanics:

- **Host Network Infrastructure Layer (ServerNetwork):** Manages passive listening ports, handles inbound connection queues via asynchronous system loops, and spawns isolated stream descriptors for each accepted client process.
- **Protocol Parsing Subsystem (GameProtocol.c / GameProtocol.h):** Decodes raw input text from client socket frames into verified instruction packets while providing data serialization to push broad state updates out over the wire .
- **Game State & Rules Engine (GameData.c / GameData.h):** Governs the structural execution rules of Texas Hold'em. This engine handles table parameters, maintains user connection indices, verifies point changes, and controls full deck manipulation routines including initial deck generation and randomization.
- **Hand Evaluation Utility Engine (HandEval.c / HandEval.h):** Operates as a pure, mathematical computation module . It accepts complex 7-card layouts from active players and calculates optimal five-card combinations while factoring in custom Anteater Face Cards and wild Jokers.



[Figure 2: Diagram of Server Module Hierarchy]

2.3 Module Interfaces

Subsystem coordination on the server follows a strict procedural escalation path to process inbound instructions and broadcast updated table parameters safely:

1. Ingress Catch: The network subsystem captures an incoming string buffer from a client file descriptor and forwards it to the controller layer.
2. Translation: The string is evaluated by ParseNetworkMessage, which validates the format and translates the text into an explicit command enum token (e.g., matching a raw stream message to a structured MSG_TYPE_ACTION flag).
3. Legality Audit: The translated packet parameters are passed to IsValidAction() within the core rules engine. This utility performs pointers-based validation checks to verify that the requesting seat index holds the active turn, the player has not folded, and their point bank covers the bet size.

4. State Transition: If approved, the rules engine commits the changes to the master data structures and shifts the game turn index.
5. Egress Broadcast: The state manager passes the updated table profile to the protocol layer's serialization engines (BuildOkMessage or general updates) before the network core broadcasts the string output to all active client socket channels simultaneously.

2.4 Overall Program Control Flow

The execution life cycle of the server process advances through a linear setup sequence before shifting to a circular gameplay loop and an interrupt-driven clean teardown:

2.4.1 Subsystem Initialization and Handshake Loop

Upon execution from the command-line interface, the server invokes its networking infrastructure:

- Socket Instantiation: Creates a stream socket endpoint using the `socket()` system call, specifying the `AF_INET` domain and `SOCK_STREAM` protocol.
- Port Binding: Attaches the socket to a configured network interface and grading port via `bind()`.
- Passive Listening: Enters listening mode via `listen()`, establishing a backlog queue to handle multiple client connection requests simultaneously.
- Connection Acceptance: The main execution thread relies on an asynchronous loop to execute `accept()`, yielding a dedicated file descriptor for each valid client connection.
- Protocol Handshake: The server blocks further game modifications for that seat until a valid ENTER protocol message containing an alias, seat index, and session password is parsed and verified. Upon validation, the server responds with an OK packet and broadcasts a table status update.

2.4.2 Core Gameplay State Machine

Once the minimum number of human players and automated Poker Bots are seated, the server manages the rules of Texas Hold'em via a circular state machine:

- `GAME_STATE_WAITING` (0): The server remains idle as long as the occupied seat count is insufficient. Automated Poker Bots can fill empty seats to satisfy start conditions.
- `GAME_STATE_PRE_FLOP` (1): The server executes `CreateDeck()` to construct the 58-card array (52 standard cards, 4 custom Anteater face cards, and 2 wild Jokers). It shuffles the deck via `ShuffleDeck()`, extracts the mandatory Antes, and invokes `DealHoleCards()` to distribute two hidden cards to each active player using sequential modulo arithmetic starting from the left of the dealer button.
- `GAME_STATE_FLOP` (2), `GAME_STATE_TURN` (3), & `GAME_STATE_RIVER` (4): The server invokes `DealCommunityCards()` to place shared cards face-up into the table structure

based on the current state boundaries. Each phase opens a sequential betting loop that processes actions (Check, Call, Raise, Fold).

- `GAME_STATE_SHOWDOWN` (5): If multiple players remain unfolded after the final betting round, the server executes `DetermineWinner()`. This step pools active hole cards with community cards, runs `EvaluateBestHand()` via the evaluation engine to handle ranking calculations, splits or awards the pot points directly to the winning player structures, shifts the dealer button index clockwise, and resets the table state to loop back to Phase 1.

2.4.3 Dynamic Exception Handling and Graceful Termination

The server continuously monitors connection stability flags. If a socket breaks or drops unexpectedly, the control layer catches the signal, updates that player's data profile to a folded state to preserve game symmetry, and closes the assigned file descriptor.

Upon receiving a termination interrupt (`SIGINT` or `SIGTERM`) from the Linux shell, the server breaks its execution loops, frees up dynamic tracking arrays, issues a closing broadcast to all open client sockets, and releases its bound system port securely.

3. Installation

3.1 System Requirements / Compatibility

3.1.1 Toolchain and Dependency Baseline

To establish a compliant development or execution host, the target environment must provide the following system packages and libraries:

- Compiler: GNU Compiler Collection (GCC) supporting the ISO C11 standard baseline (`-std=c11`).
- Build Automation: GNU Make utility for dependency tracking and compilation sequencing.
- Graphical Toolkit: GTK+ 3.0 development headers and shared libraries for client-side rendering.
- System Utilities: `pkg-config` for dynamic compiler flag injection, `glibc` standard runtime, and the standard math library (`libm`, linked via `-lm`).
- Network Infrastructure: Full socket layer compliance supporting standard IPv4 TCP/IP stream protocols.

3.1.2 Target Hardware & Subsystem Compatibility

- Processor: 64-bit x86 architecture (AMD64 / Intel 64).
- Memory: Minimum 512 MB RAM available to accommodate concurrent client instances.
- Storage: 50 MB of disk space for the compiled object files, log traces, and final binaries.
- Display Infrastructure: An active X11 display server protocol pipeline is mandatory .

Windows-based hosts must execute the system within an environment like MSYS2/UCRT64 or a Linux Virtual Machine, utilizing trusted X11 forwarding (ssh -X) coupled with a local X server client (e.g., VcXsrv or XQuartz on macOS) .

3.2 Unpacking & Configuration

To unpack the software from the source archive and compile the binaries, execute the following commands from top to bottom:

- Extract the source archive: `gtar xvzf Poker_Beta_src.tar.gz`
- Navigate to the directory: `cd Poker_Beta_src`
- Compile all components: `make all`

3.3 Building, Compilation, & Installation

3.3.1 Obtaining Source Code

First, the code must be copied/cloned into a working directory:

Example:

```
git clone <repo_url>
cd Poker_V1.0
```

If the project is distributed as a folder, the user will need to extract it and navigate accordingly.

3.3.2 Building the Program

Compilation will be handled through a makefile that will compile all required source files and link them into the final executable file. The exact command build rule will depend on the project file organization and hierarchy.

Compiler flags used will -Wall, -g to enable common warnings and for debugging purposes.

X11 will have to be handled as well for rendering and event handling in the GUI, information regarding this as stated above in earlier sections.

3.3.3 Installation Process

After following the compilation and build rules, the program can be run directly using the `./(program_name)` command. A system-wide installation may become a necessary add-on for mass client usage.

To remove compiled files and rebuild from scratch, a clean rule will be provided in the make file, such as:

```
make clean
```

This will target object files, temp files, and previous executables to make sure outdated files will not interfere with future builds.

3.3.4 Post-Installation

Following an install run, developers can verify workspace integrity on the EECS servers using this baseline sequence:

- Display Link: Confirm that an X11 tunnel is open by running `echo $DISPLAY`; it should return a valid local mapping proxy address.
- Layout Check: Launch the binary via `./bin/poker`. The application must open a stable window environment over the X connection.
- Felt Render: The 2D table drawing canvas must render the custom green poker field background cleanly with no display artifacts.
- Signal Audit: Interacting with the FOLD, CHECK, CALL, or RAISE buttons must successfully print interaction statements to the standard output buffer without causing core faults.

4. Documentation of Packages, Modules, & Interfaces

4.1 Data Structures

Card

The Card structure defines an individual card component within the custom deck footprint. The explicit inclusion of rank 14 maps to the custom Anteater Face Card, while rank 16 models the Joker. Suit N represents a suit-less card state dedicated to wildcards.

```
typedef struct {  
    char suit; // 'H', 'D', 'C', 'S', or 'N' for None  
    int rank;  // 2-10, 11(J), 12(Q), 13(K), 14(Anteater), 15(Ace), 16(Joker)  
} Card;
```

[Figure 3: Card Definition]

Deck

This data structure encapsulates the global collection of playable card components. The `DECK_SIZE` macro is defined to the integer 58, and bounding a `Card` array to 58 accommodates the standard 52-card layout supplemented by 4 custom suit-specific Anteater cards and 2 wild Jokers, eliminating dynamic allocation tracking.

```
typedef struct {  
    Card deck[DECK_SIZE];  
    int topIndex;  
} Deck;
```

[Figure 4: Deck Definition]

Player

This structure tracks connection states, point metrics, and private assets for individual participants. The macro `MAX_NAME_LEN` is defined to the integer 32.

```
typedef struct {  
    char name[MAX_NAME_LEN];  
    int seat;  
    int points;  
  
    /* Network & status management */  
    int socket;  
    unsigned char isBot;  
    unsigned char isFolded;  
  
    Card hand[2];  
} Player;
```

[Figure 5: Player Definition]

Table

This structure represents the comprehensive game board, encompassing all sub-structures. The `MAX_PLAYERS` macro is defined to the integer 8.

```
typedef struct {  
    Player players[MAX_PLAYERS];  
    Card community[5];  
    int pot;  
    int state; // 0: Waiting, 1: Pre-flop, 2: Flop, 3: Turn, 4: River, 5: Showdown  
  
    /* To track dealer and active turn */  
    int dealerIdx;  
    int activeIdx;  
} Table;
```

[Figure 6: Table Definition]

4.2 Functions & Parameters

CreateDeck

- Description - Populates the target deck array with 52 standard cards, appends 4 custom Anteater face cards, and appends 2 suit-less wild Jokers.
- Parameter(s) - Deck* pDeck - A pointer to the Deck structure that will be modified in regards to its member Card array.

ShuffleDeck

- Description - Seeds the random number generation matrix using system clock time and executes an in-place Fisher-Yates shuffle algorithm over all 58 card slots to ensure complete randomness.
- Parameter(s) - Deck* pDeck - A pointer to the Deck structure that will be modified in regards to its member Card array.

DealHoleCards

- Description - Identifies the starting position by evaluating $(pTable->dealerIdx + 1) \% MAX_PLAYERS$. Executes two discrete dealer cycles to dispense exactly two private hole cards to all seated, active players whose socket indicator does not equal -1.
- Parameter(s):
 - Table* pTable - A pointer to the global table state block.
 - Deck* pDeck - Pointer to the shuffled deck tracking framework.

DealCommunityCards

- Description - Evaluates $pTable->state$ to determine the precise community card matrix offset. Populates the target slots using cards drawn sequentially from the deck top index pointer.
- Parameter(s):
 - Table* pTable - Pointer to the global table state block.
 - Deck* pDeck - Pointer to the deck tracking framework.
 - int count - Total cards to be placed onto the board workspace.

IsValidAction

- Description - Verifies that the input request size does not exceed the user's available points pool.

If the action token matches PLAYER_ACTION_RAISE, it asserts that amount is strictly greater than currentHighest. The function will return a 1 if the move passes validation parameters; 0 if rejected as illegal.

- Parameter(s):
 - Player* pPlayer - Pointer to the player structure initiating an action.
 - int action - Action token constant identifier (PLAYER_ACTION enumeration).
 - int amount - The wager sizing passed with the instruction.
 - int currentHighest - The highest bet size committed in the active betting around.

DetermineWinner

- Description - Scans the table array to identify active, non-folded player structures. Compiles 7-card evaluation grids for each candidate by combining their 2 private down-cards with the 5 shared community cards. Delegates ranking evaluations to EvaluateBestHand() to solve hand category values. Resolves tiebreakers, divides point pools evenly across winning matches, pushes points into their balances, and sets pTable->pot to 0.
- Parameter(s) - Table* pTable - Pointer to the global table state block.

4.3 Communication Protocol

Network data transiting the TCP/IP stream socket infrastructure is serialized as explicit, human-readable ASCII text arrays terminated by a standard newline symbol (\n).

4.3.1 Client Inbound Handshake Protocol (ENTER)

Sent by a client process requesting table entry registration.

- Format: ENTER <name> SEAT <seat_number> PASSWORD <pass_phrase>\n
- Example: ENTER Vinh SEAT 2 PASSWORD AnteaterSec2026\n
- Field Assertions: <name> must not exceed 31 characters. <seat_number> must resolve between 0 and 7.

4.3.2 Server Positive Response Protocol (OK)

Returned by the host infrastructure to confirm a successful action or connection handshake.

- Format: OK SEAT=<seat_number> NAME=<name> POINTS=<current_points>\n
- Example: OK SEAT=2 NAME=Vinh POINTS=1000\n

4.3.3 Server Error Broadcast Protocol (ERROR)

Dispatched by the server engine to alert a connection that an invalid command or illegal operation was trapped.

- Format: ERROR <error_string_payload>\n
- Example: ERROR Wager size exceeds remaining point balance\n

4.3.4 Bidirectional Gameplay Action Protocol (ACTION)

Dispatched by a client to execute a wagering choice, or broadcast by the server to alert external nodes of an event change.

- Format: ACTION SEAT <seat_number> TYPE <action_enum> AMOUNT <wager_value>\n
- Example: ACTION SEAT 2 TYPE 4 AMOUNT 150\n
- Enumeration Mappings: 1 = FOLD, 2 = CHECK, 3 = CALL, 4 = RAISE.

5. Development Plan & Timeline

5.1 Task Partitions

To maximize development velocity and eliminate cross-module code modifications, the system architecture is partitioned into four decoupled development tracks. This ensures that the core rules engine can be built, verified, and linked independently into both the interactive client and the automated bot subsystems.

5.1.1 Track A: Core Mathematical Engine & Rules Boundary

This partition owns the lowest-level abstract data types and functional rules. It operates completely independently of socket connections or graphical interfaces, providing a deterministic api for upper-tier systems.

- Component Focus: GameData.h, GameData.c, HandEval.h, and HandEval.c.
- Deliverables: Fisher-Yates deck randomization routines, modulo card distribution pipelines, recursive wildcard assignment matrices for Jokers, and full Texas Hold'em split-pot scoring evaluations.

5.1.2 Track B: Authoritative Host Topology & Connection Pool

This partition manages the host system lifecycle, non-blocking networking primitives, and multi-client state coordination.

- Component Focus: ServerNetwork core infrastructure and the server main loop engine.
- Deliverables: Port binding routines, network selection loops (select/poll), file descriptor tables, connection state recovery, and automated tournament loop execution.

5.1.3 Track C: Graphical Presentation Layout & Signal Pipeline

This partition owns the visual presentation layer of the client terminal, mapping user interactions directly into engine events.

- Component Focus: GameGUI.h, GameGUI.c, and main.c.
- Deliverables: Hierarchical GTK 3.0 window structures, green poker felt 2D drawing canvases, automated widget state updates based on server broadcasts, and asynchronous signal button click mappings.

5.1.4 Track D: Protocol Serialization & Automated Agents

This partition bridges the network transport layer with internal modules through secure data serialization and builds the automated simulation testing layer.

- Component Focus: GameProtocol.h, GameProtocol.c, and the automated Poker Bot client binary.
- Deliverables: Safe text parsing drivers (snprintf, sscanf), network flag builders (ENTER, ACTION), and a non-interactive bot application running a lightweight mathematical decision matrix to fill empty table seats .

5.2 Team Member Responsibilities

Team Member	Assigned Track
Vinh Nguyen	Track A (Foundational) Track B (Foundational) Track C (Primary) Track D (Foundational)
Kun Liang	Track B (Primary) Track D (Secondary)
Jonghyun Choi	Track A (Secondary) Track D (Primary)
Hamza Faisal	Track B (Primary) Track D (Secondary)
Richard Nguyen	Track A (Foundational) Track B (Foundational) Track C (Primary) Track D (Foundational)
William Kuang	Track A (Secondary) Track D (Primary)

Back Matter

Copyright

©2026 Team T3, University of California, Irvine.

References

- <https://bicyclecards.com/how-to-play/texas-holdem-poker>
- https://oag.ca.gov/sites/all/files/agweb/pdfs/gambling/BGC_texas.pdf

Index

A	Page #
Abstract Data	3, 12-13
ACTION protocol/flag	5,6,15-16
Activeldx	7,14
Anteater Face Card (rank 14)	9,12,13
Asynchronous Processing	6,9
B	
bind()	9
BuildActionMessage	5
BuildOkMessage	9
C	
Card (Data structure)	4,7,12-13
Card.rank	12-13
Card.suit	12-13
Client-Server Architecture	3-6
ClientNetwork Module	4
connect()	6
CreateDeck()	9,14

D

DealCommunityCards()	9-10, 14-15
DealHoleCards()	9,14
DealerIdx	7,14
Deck (Data structure)	3,13
DECK_SIZE macro (=58)	13
DetermineWinner()	10,15

E

ENTER protocol msg	3,9,15-16
ERROR protocol msg	16
EvaluateBestHand()	10,15
Event-driven control flow	3,4,6

F

Fisher-Yates shuffle	14,17
FOLD	5, 15-16

G

GameData.c/GameData.h	4,7,17
GameGUI.c/.h	4,6,17
GameProtocol.c/.h	4/7/8/17-18
Game_States	9-10
GNU Make	10
GTK 3.0	3,4,6,10,17

H

HandEval.c/HandEval.h	7,17
Hand Eval Util Engine	7

Hole cards	9,10,14,15
------------	------------

I

InitializeGUI()	6
-----------------	---

In-Order Reliability (TCP)	3
----------------------------	---

IsBot-folded	13
--------------	----

IsValidAction()	8,15
-----------------	------

J

Joker	9,13,14,17
-------	------------

L

Linux	10-11
-------	-------

M

Main.c	6,17
--------	------

MAX_macros	13-14
------------	-------

Module Hierarchy	5,8
------------------	-----

N

Network Handshake	6,9
-------------------	-----

O

OK protocol message	9,16
---------------------	------

P

ParseNetworkMessage()	5,8
-----------------------	-----

Player	4,7,13
Poker Bot	6,9,17-18
Pot	7,14,15

R

RAISE	15-16
-------	-------

S

ShuffleDeck()	9,14
SIGINT/TERM	10
Socket API	3,4,9
ServerNetwork Module	7,17
Source of Truth	3,7
State	7,14

T

Table (Data structure)	4,7,13-14
Table.methods	14
Task Partitions	17-18
Texas Hold-em rules	7,9-10

U

Unit testing	3
--------------	---

V

Version 1.0	1
-------------	---

W

Wildcard handling	7,13,17
-------------------	---------

X	
---	--

X11 Display	11
-------------	----